Übung 07 – Tabellen und Daten

1000 Datenbank · Teil 3 SQL · ca. 45 Minuten · MySQL Workbench



Luigi hat Neuigkeiten

„Wir haben eine neue Kundin, zwei neue Artikel, und Aisha hat heute bestellt! Ach ja – die Pizzen werden teurer."

Ziel	Die Datenbank mit SQL erweitern, Daten einfügen, ändern und löschen – wie auf den Folien vorgemacht.
Startpunkt	`marios_pizza_start.sql` (Ordner `material` oder `datenbank/` im Repository) – **frisch ausführen**, damit alle dieselben Daten haben
Ablage	deine Lösungen in `datenbank/abfragen.sql` (mit Kommentar `-- Übung 07, Aufgabe …`)
Folien	Präsentation „1000 Datenbank" – vor der Übung erklärt und vorgemacht; bei jeder Aufgabe steht die passende Folie

Wichtig

Bearbeite die Aufgaben **der Reihe nach** – die Ergebnisse hängen voneinander ab (z. B. bekommt Aishas Bestellung die Nummer 5). Wer durcheinander kommt: Startskript neu ausführen und von vorn.

Musterlösung für einen frisch geladenen Startbestand. Die SQL-Abschnitte werden der Reihe nach ausgeführt. Angaben zu den noch folgenden Aufgaben sind erwartete Ergebnisse; die persönliche Selbstkontrolle bleibt zum eigenen Abhaken frei.

Aufgabe 1 – Startdaten laden

Hilfe: *Präsentation „1000 Datenbank", Folie 0540 (Nummer unten rechts)*

1. File › Open SQL Script… › `marios_pizza_start.sql`, alles ausführen.
2. Prüfe in einem neuen Tab: `USE marios_pizza; SELECT * FROM artikel;`

Frage	Antwort
Wie viele Artikel, Kunden und Bestellungen gibt es?	9 Artikel, 4 Kunden und 4 Bestellungen im Startbestand.
Welche artikel_id hat die Cola?	Cola hat artikel_id 7.

```
USE marios_pizza;
SELECT COUNT(*) AS anzahl_artikel FROM artikel;
SELECT COUNT(*) AS anzahl_kunden FROM kunde;
SELECT COUNT(*) AS anzahl_bestellungen FROM bestellung;
SELECT artikel_id, name FROM artikel WHERE name = 'Cola';
```

Aufgabe 2 – Struktur ändern

Hilfe: *Präsentation „1000 Datenbank“, Folie 0550–0560 (Nummer unten rechts)*

1. Füge der Tabelle `kunde` die Spalte `email` (VARCHAR(80)) hinzu.
2. Lege die Tabelle `bewertung` an: `bewertung_id` (PK, Auto Increment), `kunde_id` und `artikel_id` (beide FK, nicht leer), `sterne` (INT) und `kommentar` (VARCHAR(200)).

```
ALTER TABLE kunde ADD COLUMN email VARCHAR(80);

CREATE TABLE bewertung (
    bewertung_id INT PRIMARY KEY AUTO_INCREMENT,
    kunde_id INT NOT NULL,
    artikel_id INT NOT NULL,
    sterne INT,
    kommentar VARCHAR(200),
    FOREIGN KEY (kunde_id) REFERENCES kunde(kunde_id),
    FOREIGN KEY (artikel_id) REFERENCES artikel(artikel_id)
) ENGINE=InnoDB;

DESCRIBE kunde;
DESCRIBE bewertung;
```

bewertung_id ist der automatisch erzeugte Primärschlüssel. kunde_id und artikel_id sind Pflichtfelder und Fremdschlüssel auf vorhandene Kunden und Artikel.

Aufgabe 3 – Daten einfügen

Hilfe: *Präsentation „1000 Datenbank“, Folie 0570 (Nummer unten rechts)*

1. Füge die Kundin **Jin Park** ein (Telefon +1 718 555 0105, Henry Street 21, PLZ 11201, Brooklyn).

```
INSERT INTO kunde
    (vorname, nachname, telefon, strasse, plz, ort)
VALUES ('Jin', 'Park', '+1 718 555 0105',
        'Henry Street 21', '11201', 'Brooklyn');
```

Erwartete kunde_id: 5. email bleibt NULL, weil kein E-Mail-Wert angegeben ist.

2. Füge zwei Artikel ein: **Panna Cotta** (Dessert, 5,50 $, „Mit Beerensauce“) und **Limonade** (Getränk, 3,80 $, „0,33 l, hausgemacht“).

```
INSERT INTO artikel
    (name, beschreibung, preis, kategorie_id)
VALUES
    ('Panna Cotta', 'Mit Beerensauce', 5.50, 3),
    ('Limonade', '0,33 l, hausgemacht', 3.80, 2);
```

Erwartete artikel_id: Panna Cotta = 10, Limonade = 11. Dessert hat kategorie_id 3, Getränk hat kategorie_id 2.

Aufgabe 4 – Aishas Bestellung

Hilfe: *Präsentation „1000 Datenbank“, Folie 0580 (Nummer unten rechts)*

Aisha Johnson (kunde_id 4) hat am 29.09.2026 um 13:00 Uhr zur Abholung bestellt: 1 × Margherita und 1 × Limonade. Lege die Bestellung und die zwei Positionen an – in drei Schritten wie Luca Bianchis Bestellung auf der Folie.

> **Hinweis**
>
> Die neue `bestell_id` liefert `SELECT LAST_INSERT_ID();` direkt nach dem INSERT. Die `artikel_id` der Limonade findest du mit `SELECT * FROM artikel;`.

Schritt 1: Bestellung für Aisha anlegen.

```
INSERT INTO bestellung (kunde_id, bestelldatum, lieferart)
VALUES (4, '2026-09-29 13:00:00', 'Abholung');
```

Schritt 2: Neue Bestellnummer sofort merken und die Limonade nachschlagen.

```
SET @aisha_bestell_id = LAST_INSERT_ID();
SELECT @aisha_bestell_id AS neue_bestell_id;
SELECT artikel_id, name FROM artikel WHERE name = 'Limonade';
```

Schritt 3: Beide Positionen eintragen.

```
INSERT INTO bestellposition (bestell_id, artikel_id, menge)
VALUES (@aisha_bestell_id, 1, 1),
       (@aisha_bestell_id, 11, 1);
```

Erwartetes Ergebnis: bestell_id 5 mit zwei Positionen, je Menge 1. LAST_INSERT_ID() und die Variable gehören zur selben Verbindung wie das INSERT.

Aufgabe 5 – Ändern und löschen

Hilfe: *Präsentation „1000 Datenbank“, Folie 0590 (Nummer unten rechts)*

1. Mario erhöht den Preis **aller Pizzen** (Kategorie 1) um 0,50 $ – mit einem einzigen UPDATE. Teste vorher mit SELECT, welche Zeilen es trifft.

```
SELECT artikel_id, name, preis
FROM artikel WHERE kategorie_id = 1;

UPDATE artikel
SET preis = preis + 0.50
WHERE kategorie_id = 1;
```

Das SELECT zeigt sechs Pizzen. Alle sechs Preise steigen um 0,50; Margherita kostet anschließend 13,00. Getränkepreise bleiben gleich.

2. Aisha Johnson hat ihre Telefonnummer nachgereicht: +1 718 555 0104.

```
SELECT kunde_id, vorname, nachname, telefon
FROM kunde WHERE kunde_id = 4;

UPDATE kunde
SET telefon = '+1 718 555 0104'
WHERE kunde_id = 4;
```

3. Sam Kowalski storniert „Luigi's Verde“ (artikel_id 5) aus Bestellung 4. Lösche die passende Zeile aus `bestellposition`.

```
SELECT * FROM bestellposition
WHERE bestell_id = 4 AND artikel_id = 5;

DELETE FROM bestellposition
WHERE bestell_id = 4 AND artikel_id = 5;
```

Erwarteter Stand nach Aufgabe 5: 11 Artikel, 5 Kunden, 5 Bestellungen und 11 Bestellpositionen. Aus Bestellung 4 wird nur die Position mit artikel_id 5 entfernt.

Aufgabe 6 – Tony Russo löschen?

Hilfe: *Präsentation „1000 Datenbank“, Folie 0200, 0590 (Nummer unten rechts)*

Versuche, Tony Russo (kunde_id 1) zu löschen. Was passiert? Erkläre die Fehlermeldung mit dem Begriff aus Übung 02.

```
SELECT bestell_id FROM bestellung WHERE kunde_id = 1;
DELETE FROM kunde WHERE kunde_id = 1;
```

Erwarteter Fehler 1451: Cannot delete or update a parent row: a foreign key constraint fails.

Grund: Referenzielle Integrität. Die Bestellungen 1 und 3 verweisen über bestellung.kunde_id auf Tony Russo (kunde_id 1). Der Fremdschlüssel verhindert einen Verweis auf einen gelöschten Kunden.

Tony wird deshalb nicht gelöscht und bleibt in kunde erhalten. Diesen absichtlichen Fehlertest einzeln ausführen; danach mit der Zusatzaufgabe fortfahren.

Das kann ich jetzt

- ☐ `kunde` hat die Spalte `email`, die Tabelle `bewertung` existiert.
- ☐ Jin Park, Panna Cotta, Limonade und Aishas Bestellung sind eingetragen.
- ☐ Ich teste die WHERE-Bedingung erst mit SELECT.
- ☐ Ich kann den Fehler 1451 erklären.

Zusatzaufgabe

Sichere `bewertung.sterne` mit einer CHECK-Bedingung ab (1 bis 5 Sterne) und teste mit 7 Sternen.

```
ALTER TABLE bewertung
ADD CONSTRAINT chk_bewertung_sterne
CHECK (sterne BETWEEN 1 AND 5);
```

Negativtest: Die folgende Anweisung soll scheitern.

```
INSERT INTO bewertung (kunde_id, artikel_id, sterne, kommentar)
VALUES (4, 1, 7, 'Test mit sieben Sternen');
```

Erwartetes Ergebnis unter XAMPP/MariaDB: Fehler 4025, CONSTRAINT chk_bewertung_sterne failed. 7 liegt außerhalb von 1 bis 5; die Bewertung wird nicht gespeichert.

Zum Schluss: sichern mit Git

Speichere alles (Strg + S) und sichere deinen Stand im Repository `marios-pizza`:

```
git add .
git commit -m "SQL: Tabellen und Daten (Übung 07)"
git push
```

Kontrolle: auf github.com das Repository neu laden – dein Commit und die neuen Dateien sind zu sehen.